Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).**

I. Introduction

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confirmatory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. Related Work

Verifier-grounded NetOps evaluation and intent→config generation. Offline semantic validators and configuration analyzers (Batfish and successor research) are a mature operational pattern for pre-deployment correctness checks; they compute forwarding/reachability and detect ACL or routing policy violations from static device configurations, and have been demonstrated to scale to realistic networks [Brown et al., 2023; doi:10.1145/3603269.3604866]. Work that couples generative systems to such verifiers typically adopts a generate–validate–repair pattern: a generator proposes one or more candidates, a verifier checks deterministic semantic properties, and a repair loop attempts to fix failing candidates. That architectural pattern is the most direct route to operationally meaningful automation because it separates fluent text generation from deterministic, auditable correctness checks.

What prior LLM→NetOps evaluations vary on. Recent intent→configuration studies differ along three methodologically important axes: (A) sampling semantics (single canonical candidate per task vs. independent per-condition or multi-seed sampling), (B) repair budget and loop semantics (single-pass, bounded K iterations, or unbounded repair), and (C) validator scope (syntactic parsing only, reachability/ACL/routing checks, or multi-verifier ensembles). For example, several practical systems and empirical studies emphasize iterative repair and small fixed repair budgets, with empirical evidence that most repair gains arise in the first three iterations (see "Is Three the Magic Number? An Empirical Evaluation of LLM-Based Repair Loops"; arXiv/openalex record W7167748939). Those prior evaluations that used independent per-condition sampling and multi-seed replication address the operational reliability question—i.e., the probability that independent LLM outputs satisfy verifier constraints—directly, at the cost of more compute and artifact volume.

How verifier choice and task synthesis shape measured reliability. Two patterns recurring in the literature and in our artifact corpus are (i) template-aligned tasks that encode explicit required_sequences and per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups) which a deterministic validator checks exactly, and (ii) reuse of a single canonical candidate across multiple scoring episodes. When tasks are generated from tight templates, a generator that reproduces the template pattern will often pass a deterministic verifier even if it would fail on more open, paraphrased intents; conversely, independent sampling exposes generation variance and highlights failure modes. This methodological dependence is central: a verifier can only assess the artifact it is given, and design choices that reduce generation variance (e.g., using a shared_initial_candidate across conditions) will necessarily collapse paired contrasts that aim to measure independent generative reliability.

Artifact-anchored reproducibility as a scientific stance. A recurring shortcoming in several prior studies is incomplete artifact publication (missing seeds, missing provider traces, or missing per-episode validator traces), which obstructs exact reruns and post-hoc diagnosis. Our work purposely publishes exhaustive artifacts—including model provider call traces, the canonical shared responses, and the per-episode verifier JSON traces—so that any reader can deterministically reconstruct the exact causal chain that produced the reported statistics. Representative artifacts we publish and that directly support our diagnosis are: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), the per-episode validator trace execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...), and the full execution manifest execution/execution_manifest.json which lists every file and SHA-256 used in the run. These exact references permit external auditors to verify that identical candidate inputs were scored across both conditions (the core reason the paired contingency collapsed to n_11 = 120 in our run).

Empirical and methodological contrasts with prior work. Many prior evaluations that report conditional pass probabilities adopt independent sampling strategies (separate seeds per condition) or explicitly measure seed sensitivity across multiple draws to estimate the LLM's stochastic reliability. Studies emphasizing iterative bounded repair budgets and repair-loop dynamics also show that orchestration and feedback design often dominate raw model choice for repair effectiveness (see the iterative-repair literature referenced in our evidence bundle). By contrast, our preregistered confirmatory design chose shared_initial_candidate semantics to control for candidate content while isolating the validator behavior; that design answers a different estimand (validator consistency under identical inputs) and therefore must be interpreted differently than independent-sampling studies. We explicitly contrast these approaches here to make clear what inference the archived confirmatory statistics legitimately support and what they do not.

Contamination detection and disclosure in the literature. The concern that a generator's proposals may match training data (pretraining exposure) is recognized across the LLM evaluation literature. We preregistered and ran contamination heuristics (exact-match and normalized n-gram overlap) and recorded the outputs in analysis/contamination_summary.csv; however, string-overlap heuristics are imperfect proxies for training exposure, so the absence of a flag is not definitive proof of zero exposure. Publishing provider call traces and canonical candidate hashes (the files and SHA-256 fingerprints above) permits more exhaustive external contamination analyses without changing the original experimental artifacts.

Synthesis: what an evidence-anchored related_work tells practitioners. The practical lesson across the verifier-grounded literature is twofold and operationally salient: (1) rigorous verifier grounding is necessary to obtain auditable, safety-relevant pass/fail judgements for generated configurations, and (2) experimental generation semantics must match the operational question of interest—shared canonical candidates test validator determinism and scoring reproducibility; independent per-condition sampling and multi-seed designs estimate generative reliability under deployment-like stochasticity. Our artifact bundle is intentionally complete so that other researchers can replay either paradigm (shared_initial or independent_per_condition) deterministically and thereby answer the specific operational question they care about without ambiguity.

III. Methodology

Preregistration and frozen design. The full preregistration (CAND-2026-001-NetConfig-Semantic-v1) was archived prior to any model calls (SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a3 The preregistered confirmatory design specified a paired comparison across N = 120 tasks (40 tasks per topology class: edge, datacenter, multi-AS), the primary estimand paired_success_rate_difference_guarded_minus_baseline, confirmatory generation_semantics = "shared_initial_candidate", one canonical seed_0 per pair for the confirmatory analysis, the deterministic validator deterministic_netops_validator_v5 (validator_version 5.0) and the scoring rule full_intent_constraint_validation. The preregistered analysis executor was paired_binary_analysis_v1 with bootstrap inference (10,000 resamples, seed = 1729). The preregistration and analysis plan (including failure treatment, retry policy, and contamination heuristics) are published in the artifact bundle.

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: the initial_state, expected_final_state, an explicit required_sequence (the minimal safe command sequence), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation (changed_interface_count, transient_rule_count). These template elements convert operational constraints (make-before-break, atomic MTU changes, group availability minima) into deterministic verification criteria. Representative task manifests and their exact file locations are archived (execution/task_manifest.jsonl; see artifact_examples within the bundle).

Model, orchestration, and exact generation semantics. The declared LLM in the preregistration and execution manifest was gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the pinned model configuration file is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Per the preregistered confirmatory semantics we produced one canonical candidate per task (shared_initial_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; these canonical shared candidates are archived under execution/responses/*-shared-initial.txt (example SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...). The execution manifest records that model_calls_used = 120 while planned_episode_count = 240 (the shared_initial design reduces distinct generation calls per condition and is explicitly documented in the manifest).

Deterministic validation and scoring. Each candidate was evaluated by deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline performs: normalization and parsing of candidate commands; enforcement of per-task workflow_policies and transient constraints (e.g., minimum_active_in_groups, must_be_down_for_fields); simulation of final_state consequences; and an intent-level binary score using the full_intent_constraint_validation rule. Per-episode JSON scoring traces (execution/scoring/*.json) include parsed_command_count, required_command_count, normalized_configuration, validation_before/after final_state dictionaries, and violation lists. Representative scoring files are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. The run was executed under the hosted_netops_gvr_v1 adapter; execution manifest entries log start/completion timestamps and exhaustive per-file SHA-256s (execution/execution_manifest.json). Planned_episode_count = 240; completed_episode_count = 240; model_calls_used = 120 (shared_initial generation per pair counted once); failed_episode_count = 0. A preregistered retry policy allowed up to two automatic retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run per preregistration and recorded in analysis/contamination_summary.csv; no confirmatory items were flagged under the chosen thresholds (however the preregistration and Disclosure explicitly note that absence of a flag is not proof of zero pretraining exposure).

Primary inferential procedure. For each pair i (i = 1..120) the baseline_i and guarded_i binary verifier outcomes were recorded. The paired estimator is mean_{i}(guarded_i - baseline_i). Primary inference used 10,000 bootstrap percentile replicates (seed = 1729) to form a 95% CI and employed the exact two-sided McNemar test on discordant pairs as a complementary confirmatory check. All analysis code, bootstrap parameters, and the analysis log are archived under analysis/ (see analysis/analysis_log.jsonl SHA-256 ff20bc07...).

Key artifact index (representative, for rapid audit). The following canonical artifact paths and SHA-256 fingerprints are included in the submission bundle to enable byte-for-byte reruns and immediate inspection: - preregistration: provenance/preregistration_CAND-2026-001.json (SHA-256 7db1663cf3982c8e...) - initial master prompt: provenance/master_prompt.txt (SHA-256 1872df1e1805d2d9...) - model configuration: execution/model_configuration.json (SHA-256 a40e96e212ab87da...) - execution manifest (full artifact index): execution/execution_manifest.json - shared canonical candidate (example): execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...) - scoring trace (example): execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) - analysis artifacts: analysis/condition_summary.csv (SHA-256 9c77c96f37c4b6ff...), analysis/paired_contingency_table.csv (SHA-256 9f25790c7eeaafb3...).

Deviations and transparency. The methodology above follows the frozen preregistration; all deviations, retries, exclusions, and per-file SHA-256 fingerprints are recorded in the execution manifest and analysis logs. The artifact bundle provides the exact provenance needed for external auditors to re-execute the analysis under alternative generation semantics (e.g., independent_per_condition or multi-seed confirmatory designs) without introducing unspecified choices.

IV. Results

Confirmatory counts and inferential summary (artifact grounding). All confirmatory scoring episodes completed and are traceable in the execution manifest (execution/execution_manifest.json). The archived confirmatory counts are: baseline_success_count = 120/120; guarded_success_count = 120/120; complete_pair_count = 120. The paired contingency table is degenerate: n_11 = 120, n_10 = 0, n_01 = 0, n_00 = 0 (analysis/paired_contingency_table.csv SHA-256 9f25790c7eeaafb3...). The primary estimator (mean of guarded_i - baseline_i) = 0.00. The preregistered bootstrap (10,000 resamples, seed 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided p = 1.0. Analysis artifacts (condition_summary.csv SHA-256 9c77c96f37c4b6ff..., paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7...) are published in the bundle.

Representative validator traces (artifact examples). The per-episode JSON traces record parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after final_state dictionaries, and violation_count. Examples: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) shows parsed_command_count = 4, required_command_count = 4, violation_count = 0 and contains explicit before/after final_state mappings. The canonical candidate used for that pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Multiple pairs exhibit identical baseline/guarded/shared response fingerprints (e.g., task-000003 artifacts all share SHA-256 f7f4db249ecbbce...), demonstrating that identical candidates were evaluated across both conditions.

Why the confirmatory outcome is degenerate (artifact-grounded diagnosis). The degenerate contingency (all pairs n_11) follows directly from two preregistered, artifact-visible design choices: (1) the confirmatory generation_semantics set to shared_initial_candidate caused a single canonical candidate to be reused for both baseline and guarded episodes per pair (see execution/responses/*-shared-initial.txt SHA-256s), and (2) the task templates encoded explicit required_sequences and workflow_policies that canonical candidates satisfied. Under these conditions a deterministic validator necessarily returns identical pass/fail judgements for identical inputs, hence the paired difference must be zero. The execution and scoring artifacts (execution/responses/* and execution/scoring/*) fully document this chain of causation and are published to enable exact reruns.

Contamination and exposure. Preregistered contamination heuristics flagged no confirmatory items under the chosen thresholds (analysis/contamination_summary.csv). All provider call traces and call_cache entries are published to permit stronger external exposure checks; absence of a flag in the preregistered heuristic is not proof of zero pretraining exposure and is explicitly noted in the Disclosure.

V. Discussion

Interpretation of confirmatory inference. The reported confirmatory statistics are correct for the preregistered estimand: they test whether a deterministic verifier treats identical artifacts differently under two scoring conditions. They do not, however, estimate the operational quantity of independent LLM generative reliability (the probability that independently sampled LLM candidates satisfy verifier constraints). For deployment decisions operators need independent_per_condition pass probabilities; the archived artifacts permit reruns under those semantics without creating new task definitions.

Artifact-grounded remediation and runnable next experiments. Using the published bundle one can immediately perform informative reruns while preserving reproducibility: (A) independent_per_condition confirmatory sampling — generate separate seeds per condition per task (e.g., seed_0 baseline, seed_1 guarded), rescore with deterministic_netops_validator_v5 and compute paired contrasts; (B) multi-seed confirmatory design — draw K >= 3 independent seeds per condition per task and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — add a second independent verifier (e.g., reachability emulator) and report intersection/union pass rates to reduce validator–task coupling. Because the generator, provider traces, and scoring harness are fully archived (execution/model_configuration.json SHA-256 a40e96e2..., execution/provider_calls/*.json, execution/scoring/*.json), these reruns are deterministic and reproducible.

Threats to validity (artifact-identified). Internal validity: the shared_initial design intentionally removed generation variance and therefore precluded testing independent generative reliability. Construct validity: template-aligned tasks and a single deterministic validator increase the chance that canonical candidates satisfy required_sequences, possibly overstating success in open distributions. External validity: a single declared model (gpt-5-mini) and synthetic tasks limit generalization to other model families, open natural language intents, and production topologies. Conclusion validity: a degenerate contingency table produces trivial but correct inferential outputs; interpreting them beyond their precise estimand would be inappropriate. All these threats are documented in the published traces.

Operational implications for NetOps practice. Our artifact-anchored diagnosis shows that (1) verifier grounding is necessary but not sufficient—experimental semantics must match the deployment question; (2) transparent preregistration plus exhaustive artifact publication materially clarifies what a given confirmatory test measures; and (3) simple rerun protocols (independent sampling, multi-seed replication, validator diversity) convert a degenerate confirmatory design into an informative operational comparison without sacrificing reproducibility.

VI. Limitations

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.

- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

VII. Conclusion

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all N = 120 tasks (both 120/120). Archived evidence (shared_initial candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

Disclosure Statement

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37. Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

References

[1] https://doi.org/10.1145/3603269.3604866
[2] https://doi.org/10.23919/cnsm62983.2024.10814407
[3] https://doi.org/10.1002/nem.70029